Programación de aplicaciones

Nombre: _____ Fecha: _____

Qué vas a aprender

- Entender qué es una aplicación y cómo se organiza: interfaz y lógica
- Conocer el ciclo de desarrollo de una app
- Crear una aplicación sencilla para móvil con App Inventor
- Aplicar criterios de usabilidad, accesibilidad, privacidad y seguridad

Para pensar

Si usas decenas de apps cada día, ¿podrías crear una tú mismo en una clase?

Explicación

¿Qué es una aplicación?

Una **aplicación** (*app*) es un programa diseñado para una tarea concreta: mensajería, juegos, mapas, banca... Según dónde funcionan, hay:

- **Apps nativas**: se instalan desde una tienda y se programan para un sistema concreto, **Android** o **iOS**. Aprovechan bien la cámara, el GPS o los sensores.
- **Apps web**: funcionan en el navegador, sin instalar, en cualquier dispositivo.
- **Apps de escritorio**: para ordenadores (procesadores de texto, editores).

Toda app tiene dos partes: la **interfaz** (lo que ves y tocas) y la **lógica** (el programa que decide qué pasa). Detrás, muchas usan además un **servidor** o una base de datos en la nube.

undefined

El ciclo de desarrollo

Crear una app es un proyecto con fases, que se repiten varias veces hasta que funciona bien:

1. **Idea y análisis**: ¿qué problema resuelve?, ¿quién la usará?
2. **Diseño**: bocetos de cada pantalla (*wireframes*) y flujo de navegación.
3. **Programación**: interfaz y lógica.
4. **Pruebas**: en el simulador y en móviles reales, con usuarios.
5. **Publicación**: en una tienda de aplicaciones o en la web.
6. **Mantenimiento**: corregir errores y añadir mejoras (las actualizaciones).

Las empresas trabajan con **metodologías ágiles**: versiones pequeñas que se prueban y mejoran cada pocas semanas.



App Inventor

MIT App Inventor es un entorno web gratuito para crear apps de Android (y algunas de iOS) con **programación por bloques**, como Scratch. Tiene dos vistas:

- **Diseñador**: arrastras **componentes** a la pantalla del móvil y ajustas sus **propiedades** (texto, color, tamaño).
- **Bloques**: programas qué ocurre cuando el usuario interactúa con esos componentes.

Para probar la app al momento se usa la aplicación **MIT AI2 Companion** en el móvil, que se conecta escaneando un código QR. Al terminar, se genera un archivo `.apk` para instalarla.

undefined

Componentes y propiedades

Componente	Para qué sirve	Propiedades típicas
Botón	que el usuario toque	Texto, ColorDeFondo, Imagen
Etiqueta	mostrar texto	Texto, TamañoDeLetra
Caja de texto	que el usuario escriba	Pista, SoloNúmeros
Imagen	mostrar una foto o dibujo	Foto, Ancho, Alto
Disposiciones	ordenar en filas o columnas	Alineación
No visibles	TextoAVoz, Sonido, Acelerómetro, Cámara, Reloj, TinyDB	según el componente

Los componentes **no visibles** no aparecen en pantalla pero dan acceso a las funciones del móvil: hablar, vibrar, medir la inclinación, guardar datos...

undefined

Programar con eventos

Las apps funcionan por **eventos**: no hacen nada hasta que pasa algo (el usuario toca un botón, agita el móvil, llega un mensaje). Cada evento tiene su bloque:

```
cuando BotónHablar.Clic
    llamar TextoAVoz1.Hablar
        mensaje: CajaTexto1.Texto
```

Este programa lee en voz alta lo que el usuario haya escrito. Otros eventos útiles: **Pantalla1.Inicializar** (al abrir la app), **Acelerómetro.Agitado**, **Reloj.Temporizador** (cada cierto tiempo). Dentro de los eventos se usan variables, condicionales y bucles, exactamente igual que en Scratch.



Guardar datos y usar varias pantallas

Si cierras una app, sus variables se borran. Para que los datos **persistan** (una lista de tareas, la mejor puntuación) se guardan en una base de datos:

- **TinyDB**: guarda datos en el propio móvil con una **etiqueta** y un **valor**.
- **Bases de datos en la nube**: comparten datos entre varios usuarios, como en un chat.

Las apps más completas tienen **varias pantallas** (inicio, juego, ajustes). Se pasa de una a otra con el bloque «abrir otra pantalla». Antes de programarlas, dibuja el **mapa de navegación**: qué pantallas hay y qué botón lleva a cada una.

undefined

Usabilidad y accesibilidad

Una app técnicamente perfecta fracasa si es difícil de usar. La **usabilidad** mide lo fácil que resulta:

- Botones **grandes** y separados (al menos el tamaño de la yema de un dedo).
- Textos **claros** y breves; iconos conocidos.
- **Coherencia**: los mismos colores y posiciones en todas las pantallas.
- **Respuesta**: cuando el usuario toca algo, debe notar que ha pasado algo.

La **accesibilidad** busca que todo el mundo pueda usarla: buen contraste, letra ampliable, descripciones de las imágenes para los lectores de pantalla y no depender solo del color para dar información.

undefined

Privacidad, seguridad y ética

Las apps pueden pedir **permisos**: cámara, micrófono, ubicación, contactos. Como usuario, concede solo los imprescindibles. Como programador, pide solo los que necesites y explica para qué.

- Protege los datos personales de los usuarios; en Europa lo exige el **Reglamento General de Protección de Datos** (RGPD).
- Descarga apps solo de tiendas oficiales.
- Respeta los **derechos de autor** de imágenes y sonidos que uses.
- Evita diseños que enganchen o manipulen, como notificaciones constantes.

Una buena app resuelve un problema real **sin aprovecharse** de quien la usa.

undefined

Otras herramientas y lenguajes

App Inventor es ideal para aprender. Otras opciones:

- **Thunkable** y **Kodular**: también por bloques, con más componentes.
- **Code.org App Lab**: apps web con bloques o JavaScript.
- Profesionales: **Android Studio** (Kotlin o Java) para Android, **Xcode** (Swift) para iOS, o **Flutter** y React Native para ambas a la vez.

La lógica que has aprendido (eventos, variables, condicionales, bucles, listas) es la misma en todas. Lo que cambia es la forma de escribirla.

undefined

Crear una app

Interfaz + lógica	lo que se ve · lo que decide el programa
Ciclo	idea → diseño → programar → probar → publicar
App Inventor	diseñador (componentes) + bloques (eventos)
Datos	variables · TinyDB · varias pantallas
Buenas prácticas	usable · accesible · pocos permisos

Ejercicios

1. **Explica la diferencia entre una app nativa y una app web.**

2. **Ordena las fases: pruebas, idea, publicación, programación, diseño.**

3. **¿Qué componentes necesitas para una app que convierta euros a dólares?**

4. **Escribe el bloque para la app anterior con un cambio de 1,08.**

5. **¿Qué componente no visible usarías para una app que cuente pasos? ¿Y para una linterna?**

6. **Nombra cuatro criterios de usabilidad.**

7. **Nombra tres medidas de accesibilidad.**

8. **Una app de linterna pide acceso a tus contactos y ubicación. ¿Qué harías?**

9. **Dibuja el mapa de navegación de una app de recetas con tres pantallas.**

10. **Diseña y programa una app de lista de tareas que guarde los datos.**

Practica

1. **Las dos partes de toda app son...**

   A) pantalla y batería
   B) interfaz y lógica
   C) icono y nombre

2. **Una app que funciona en el navegador sin instalar es una app...**

   A) nativa
   B) de escritorio
   C) web

3. **App Inventor fue creado por el...**

   A) MIT
   B) BBC
   C) Apple

4. **En App Inventor, los botones y etiquetas se colocan en el...**

   A) editor de bloques
   B) Companion
   C) Diseñador

5. **El componente que lee un texto en voz alta es...**

   A) Etiqueta
   B) TextoAVoz
   C) TinyDB

6. **Para guardar datos en el móvil aunque se cierre la app se usa...**

   A) TinyDB
   B) una variable global
   C) una etiqueta

7. **Las apps funcionan principalmente por...**

   A) orden alfabético
   B) eventos
   C) azar

8. **Que una app la pueda usar una persona con baja visión es...**

   A) rendimiento

B) compilación

C) accesibilidad

9. **El boceto de una pantalla se llama...**

A) wireframe

B) apk

C) sprite

10. **Como programador, debes pedir...**

A) todos los permisos

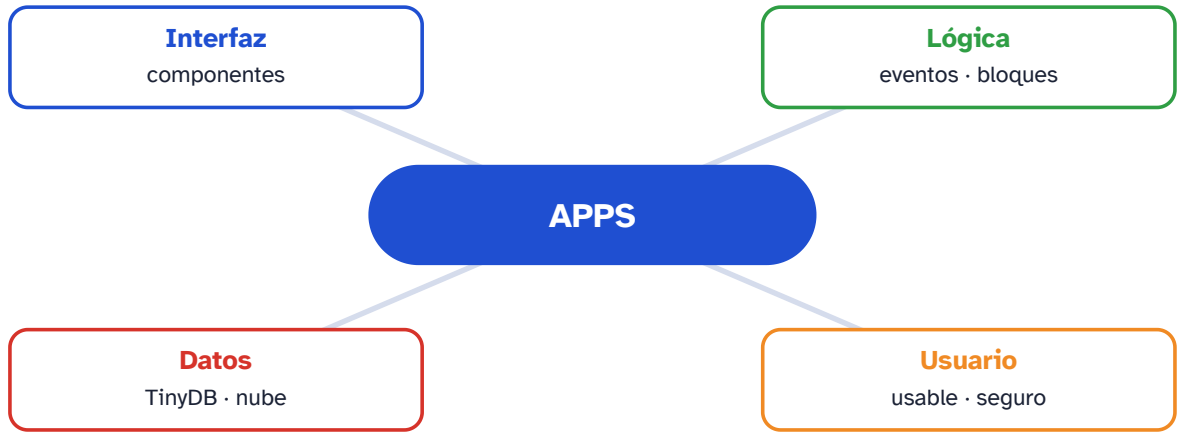
B) solo los permisos necesarios

C) ningún dato nunca

Explícalo con tus palabras

Incluye: Qué diferencia hay entre interfaz y lógica; Las fases del desarrollo de una app; Cómo se programa un botón en App Inventor; Qué hace usable y accesible una aplicación.

Resumen



Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

App nativa

Se instala y se programa para Android o iOS.

Interfaz

La parte de la app que se ve y se toca.

Componente

Elemento de la app: botón, etiqueta, sensor...

Propiedad

Característica de un componente: texto, color, tamaño.

Evento

Algo que ocurre y dispara un programa: un clic, agitar.

TinyDB

Base de datos en el móvil para guardar datos.

Usabilidad

Facilidad de uso de una aplicación.

Permiso

Acceso que pide una app a cámara, ubicación, etc.

Soluciones

1. **B)** Lo que se ve y lo que decide.
2. **C)** Cualquier dispositivo.
3. **A)** Como Scratch.
4. **C)** Vista de diseño.
5. **B)** No visible.
6. **A)** Etiqueta + valor.
7. **B)** Responden a lo que pasa.
8. **C)** Para todo el mundo.
9. **A)** Esquema de diseño.
10. **B)** Y explicar para qué.

Soluciones de los ejercicios

1. La nativa se instala y se programa para un sistema; la web funciona en el navegador sin instalar.
2. Idea, diseño, programación, pruebas, publicación (y mantenimiento).
3. Caja de texto (cantidad), botón (convertir) y etiqueta (resultado).
4. cuando Botón.Clic → poner Etiqueta.Texto a CajaTexto.Texto · 1,08.
5. Podómetro (o acelerómetro); para la linterna, el componente de la cámara con flash o la pantalla en blanco.
6. Botones grandes, textos claros, coherencia visual, respuesta a las acciones.
7. Buen contraste, letra ampliable, descripción de imágenes para lectores de pantalla.
8. Desconfiar y no instalarla o denegar esos permisos: no los necesita.
9. Inicio → lista de recetas → detalle de la receta, con botón de volver.
10. Respuesta abierta; caja de texto, botón añadir, vista de lista y TinyDB para guardar la lista.